

Week 2 , DAY-1 Hands-On Activities – Keerthivasan R

Problem 1: Personal Notes Saver using LocalStorage (Level-1)

Scenario

You are building a simple web page where users can write daily notes and save them in their browser without using a server.

✦ Requirements

- A textarea for writing notes.
- A **Save** button (using onclick).
- A **Clear** button.
- Notes must:

Be stored in localStorage

Automatically load when the page refreshes

- Display stored note on page load.

🔧 Technical Constraints

- Must use:
 - onclick inline event
 - localStorage.setItem()
 - localStorage.getItem()
 - localStorage.removeItem()
- No backend/database.
- Pure HTML + JavaScript only.
- Data stored as key-value pair.

🎯 Learning Outcome

You should be able to:

- Inline event handling (onclick)
- Browser Storage APIs
- Storing & retrieving key-value data
- Page load event handling
- Basic DOM manipulation

CODE:

```
<!DOCTYPE html>
<html>

<head>
  <!-- Title shown in browser tab -->
  <title>Personal Notes Saver</title>
</head>

<!-- onload calls loadNote() automatically when page opens -->

<body onload="loadNote()">

  <!-- Page Heading -->
  <h2>Daily Notes</h2>

  <!-- Textarea where user writes notes -->
  <!-- id is used to access this element in JavaScript -->
  <textarea id="noteArea" rows="8" cols="40"></textarea>
  <br><br>

  <!-- Button to save note (calls saveNote function) -->
  <button onclick="saveNote()">Save</button>

  <!-- Button to clear note (calls clearNote function) -->
  <button onclick="clearNote()">Clear</button>

  <script>

    // Function to save note in browser storage
    function saveNote() {

      // Get the text entered in textarea
      let note = document.getElementById("noteArea").value;

      // localStorage stores data in browser permanently
      // "dailyNote" is the key, note is the value
      localStorage.setItem("dailyNote", note);

      // Show confirmation message
      alert("Note Saved!");
    }

    // Function runs automatically when page loads
    function loadNote() {

      // Retrieve saved note using the same key
      let savedNote = localStorage.getItem("dailyNote");

      // If there is a saved note, display it in textarea
      if (savedNote) {
        document.getElementById("noteArea").value = savedNote;
      }
    }
  }
</script>

```

```

// Function to delete saved note
function clearNote() {

    // Remove note from localStorage
    localStorage.removeItem("dailyNote");

    // Clear textarea content
    document.getElementById("noteArea").value = "";

    // Show confirmation message
    alert("Note Cleared!");
}

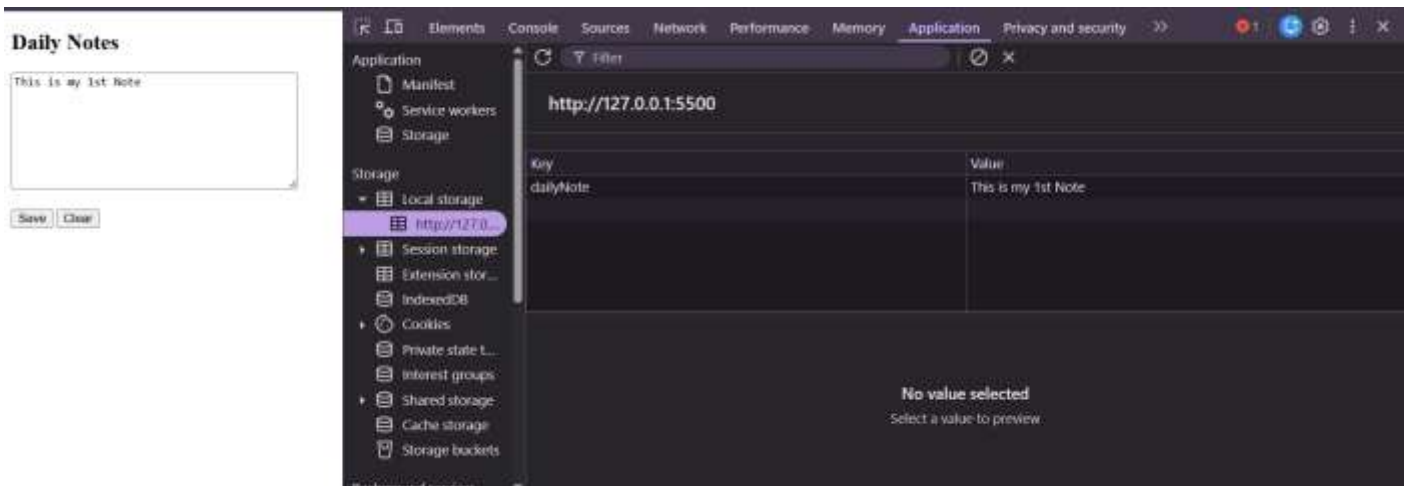
</script>

</body>

</html>

```

OUTPUT:



Problem 2: Live Form Validation with Events (Level-1)

Scenario

Create a simple registration form that validates user input when fields change.

🔴 Requirements

- Fields:
 - Name
 - Email
 - Age
- Use:
 - onchange
 - onclick
- Validate:
 - Name cannot be empty
 - Email must contain "@"
 - Age must be greater than 18
- Display validation message dynamically.
- Store valid user data in sessionStorage.

🔧 Technical Constraints

- Must use inline onchange events.
- Store valid data using:
 - `sessionStorage.setItem()`
- No external libraries.
- Use basic JavaScript only.

🎯 Learning Outcome

You will be able to:

- onchange event usage
- Form validation logic
- Difference between localStorage and sessionStorage
- Dynamic DOM updates

CODE:

```
<!DOCTYPE html>
<html>

<head>
  <!-- Title shown in the browser tab -->
  <title>Registration Form</title>
</head>

<body>

  <!-- Form heading -->
  <h2>Registration Form</h2>

  <!-- Name input field -->
  <!-- onchange triggers validation when user leaves the field -->
  Name: <input type="text" id="name" onchange="validateName()"><br><br>

  <!-- Email input field -->
  Email: <input type="text" id="email" onchange="validateEmail()"><br><br>

  <!-- Age input field -->
  Age: <input type="number" id="age" onchange="validateAge()"><br><br>

  <!-- Submit button -->
  <!-- Calls submitForm() when clicked -->
  <button onclick="submitForm()">Submit</button>

  <!-- Paragraph to show validation error messages -->
  <p id="message" style="color:red;"></p>

  <script>

    /*
      Boolean flags to track whether each field is valid.
      These help ensure form is submitted ONLY when all fields are correct.
    */
    let isValidName = false;
    let isValidEmail = false;
    let isValidAge = false;

    // Function to validate Name field
    function validateName() {

      // Get value entered in name input
      let name = document.getElementById("name").value;

      // trim() removes extra spaces
      if (name.trim() === "") {
        // Show error if empty
        document.getElementById("message").innerText = "Name cannot be empty";
        isValidName = false;
      } else {
        // Clear error if valid
      }
    }
  </script>
</body>
</html>
```

```

        document.getElementById("message").innerText = "";
        isValidName = true;
    }
}

// Function to validate Email field
function validateEmail() {

    // Get email input value
    let email = document.getElementById("email").value;

    // Simple validation: email must contain '@'
    if (!email.includes("@")) {
        document.getElementById("message").innerText = "Invalid email";
        isValidEmail = false;
    } else {
        document.getElementById("message").innerText = "";
        isValidEmail = true;
    }
}

// Function to validate Age field
function validateAge() {

    // Get age value
    let age = document.getElementById("age").value;

    // Age condition check
    if (age <= 18) {
        document.getElementById("message").innerText = "Age must be greater than
18";

        isValidAge = false;
    } else {
        document.getElementById("message").innerText = "";
        isValidAge = true;
    }
}

// Function executed when Submit button is clicked
function submitForm() {

    // Check if all validations passed
    if (isValidName && isValidEmail && isValidAge) {

        // Fetch values from input fields
        let nameValue = document.getElementById("name").value;
        let emailValue = document.getElementById("email").value;
        let ageValue = document.getElementById("age").value;

        // Create object to store user data
        let user = {
            name: nameValue,
            email: emailValue,
            age: ageValue
        };
    }
}

```

```

        /*
        sessionStorage stores data temporarily.
        Data is removed when browser tab is closed.
        JSON.stringify converts object into string format.
        */
        sessionStorage.setItem("userData", JSON.stringify(user));

        // Success message
        alert("Data Saved in Session Storage!");

    } else {
        // Show error if validation failed
        document.getElementById("message").innerText =
            "Please fix errors before submitting.";
    }
}

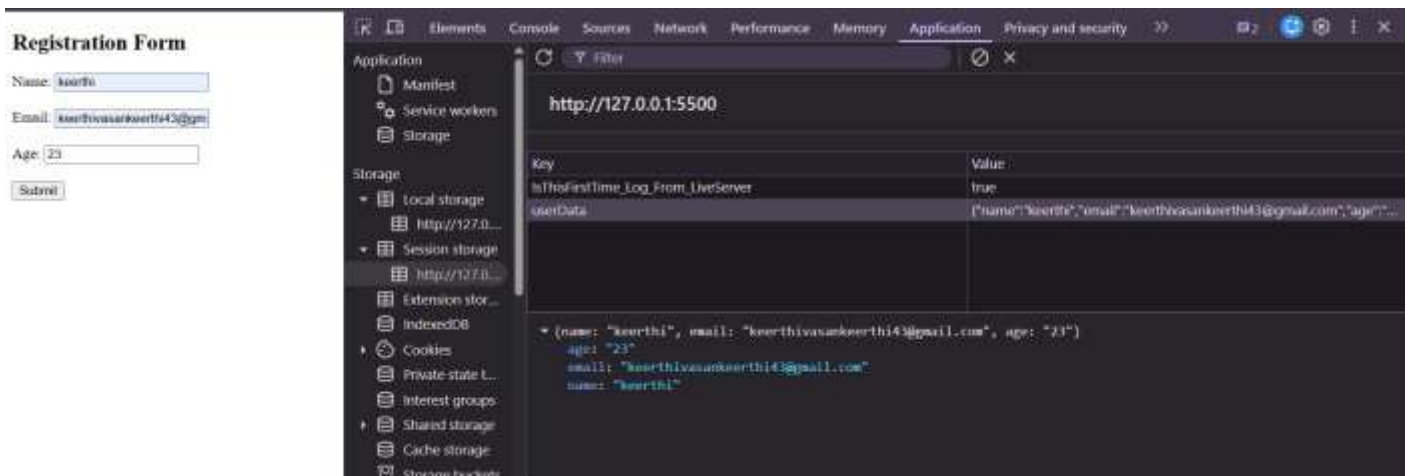
</script>

</body>

</html>

```

OUTPUT:



Problem 3: Location-Based Weather Logger (Level-2)

Scenario

Create a web application that fetches the user's geographic location and stores location history in localStorage.

✦ Requirements

- Button: **Get My Location**
- Use:
`navigator.geolocation.getCurrentPosition()`
- Display:
Latitude
Longitude
- Handle:
Permission denied
Timeout
Location unavailable
- Save last 5 location entries in localStorage.
- Display location history on page load.

🔧 Technical Constraints

- Must handle:
Success callback
Error callback
- Use browser permission handling.
- Store data as JSON using:
`JSON.stringify()`
`JSON.parse()`
- Use inline event (onclick).

🎯 Learning Outcome

Learners should be able to:

- Geolocation API
- Handling browser permissions
- Error handling in APIs
- Managing structured data in localStorage
- JSON parsing and stringifying

CODE:

```
<!DOCTYPE html>
<html>

<head>
  <!-- Title shown in browser tab -->
  <title>Location Logger</title>
</head>

<!-- loadHistory() runs automatically when page loads -->
<body onload="loadHistory()">

  <h2>Location Logger</h2>

  <!-- Button to get current location -->
  <button onclick="getLocation()">Get My Location</button>

  <!-- Paragraph to display current location or error -->
  <p id="output"></p>

  <h3>Location History</h3>

  <!-- List to display previous locations -->
  <ul id="history"></ul>

  <script>

    // Function called when button is clicked
    function getLocation() {

      /*
       navigator.geolocation is a browser API
       getCurrentPosition() gets the user's current location.
       It takes two callbacks:
       1. success → if location is retrieved
       2. error → if permission denied or failed
      */
      navigator.geolocation.getCurrentPosition(success, error);
    }

    // Runs if location is successfully retrieved
    function success(position) {

      // Extract latitude and longitude from position object
      let lat = position.coords.latitude;
      let lon = position.coords.longitude;

      // Display current coordinates on screen
      document.getElementById("output").innerText =
        "Latitude: " + lat + " | Longitude: " + lon;

      /*
       Get stored locations from localStorage.
       JSON.parse converts string back to array.
      */
    }
  </script>

```

```

    If no data exists, use empty array [].
    */
    let locations = JSON.parse(localStorage.getItem("locations")) || [];

    // Add new location object to array
    locations.push({
        lat,
        lon,
        time: new Date().toLocaleString() // store current date & time
    });

    // Keep only last 5 locations
    if (locations.length > 5) {
        locations.shift(); // removes oldest entry
    }

    // Convert array to string and store in localStorage
    localStorage.setItem("locations", JSON.stringify(locations));

    // Reload history list on screen
    loadHistory();
}

// Runs if there is an error (e.g., permission denied)
function error(err) {

    // Display error message
    document.getElementById("output").innerText =
        "Error: " + err.message;
}

// Loads and displays saved location history
function loadHistory() {

    // Retrieve saved locations from storage
    let locations = JSON.parse(localStorage.getItem("locations")) || [];

    let historyList = document.getElementById("history");

    // Clear existing list before reloading
    historyList.innerHTML = "";

    // Loop through each saved location
    locations.forEach(loc => {

        // Create new list item
        let li = document.createElement("li");

        // Display latitude, longitude and time
        li.innerText = `${loc.lat}, ${loc.lon} (${loc.time})`;

        // Add item to unordered list
        historyList.appendChild(li);
    });
}

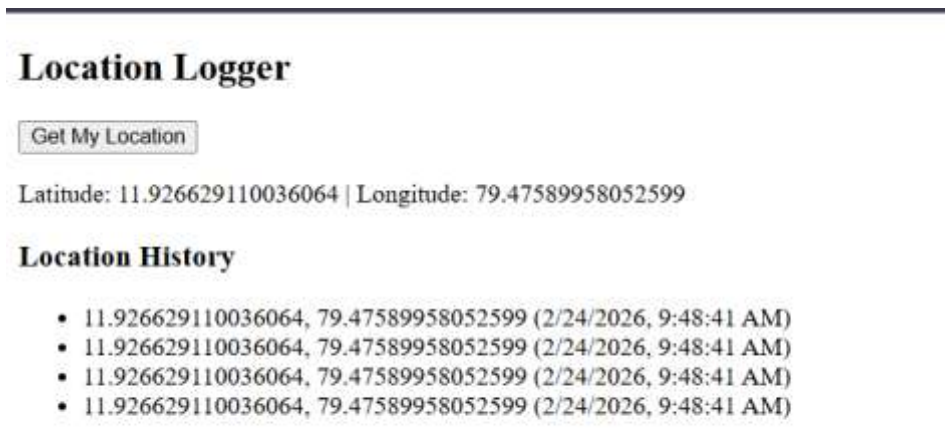
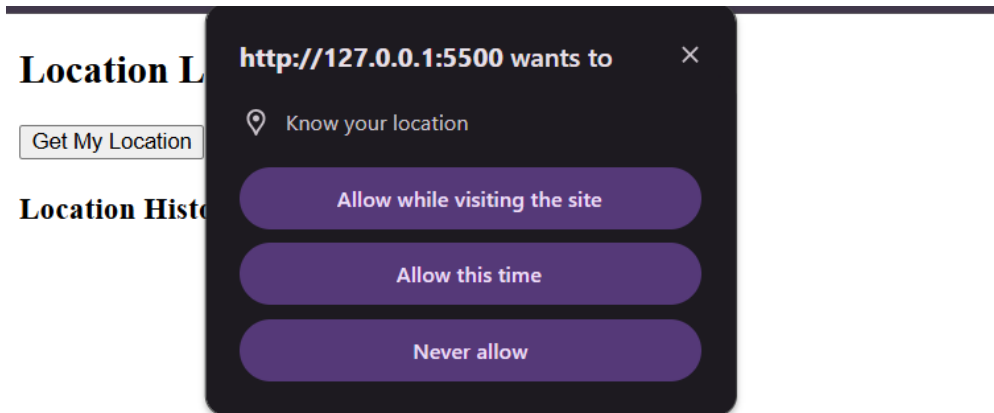
```

```
</script>

</body>

</html>
```

OUTPUT:



Problem 4: Mini Expense Tracker using Client-Side Database (Level-2)

Scenario

Develop a client-side expense tracker where users can add, view, and delete expenses using a browser-supported database.

🔴 Requirements

1. Fields:
 - Expense Title
 - Amount
 - Date

2. Buttons:

- Add Expense
- View Expenses
- Delete Expense

3. Use:

- Client-side database (Web SQL or IndexedDB)

4. Execute SQL-like queries:

- CREATE TABLE
- INSERT
- SELECT
- DELETE

5. Maintain transaction handling.

6. Display expense list dynamically.

Technical Constraints

- Must use:
 - Client-side DB transactions
 - SQL execution methods
- Must handle:
 - Transaction errors
 - Query errors
- No server/database allowed.
- Pure JavaScript + HTML.

Learning Outcome

You will be able to:

- Client-side database concepts
- Executing SQL queries in browser
- Managing transactions
- Advanced DOM rendering
- Persistent structured storage

CODE:

```
<!DOCTYPE html>
```

```
<html>

<head>
  <!-- Title shown in browser tab -->
  <title>Mini Expense Tracker (IndexedDB)</title>

  <!-- Basic styling for better UI appearance -->
  <style>
    body {
      font-family: Arial;
      background-color: #f4f4f4;
      padding: 30px;
    }

    h2 {
      margin-bottom: 20px;
    }

    input {
      padding: 6px;
      margin-right: 10px;
    }

    button {
      padding: 6px 12px;
      cursor: pointer;
    }

    table {
      margin-top: 20px;
      border-collapse: collapse;
      width: 80%;
      background: white;
    }

    th,
    td {
      border: 1px solid #999;
      padding: 10px;
      text-align: center;
    }

    th {
      background-color: #ddd;
    }
  </style>
</head>

<body>

  <h2>Mini Expense Tracker (IndexedDB)</h2>

  <!-- Input fields to collect expense details -->
  <input type="text" id="title" placeholder="Expense Title">
  <input type="number" id="amount" placeholder="Amount">
```

```

<input type="date" id="date">

<br><br>

<!-- Buttons to trigger add and view operations -->
<button onclick="addExpense()">Add Expense</button>
<button onclick="viewExpenses()">View Expenses</button>

<h3>Expense List</h3>

<!-- Table to display expenses -->
<table id="expenseTable">
  <thead>
    <tr>
      <th>ID</th>
      <th>Title</th>
      <th>Amount</th>
      <th>Date</th>
      <th>Action</th>
    </tr>
  </thead>
  <tbody></tbody>
</table>

<script>

  // Variable to store database reference
  let db;

  /*
   Open IndexedDB database
   "ExpenseDB" → database name
   1 → version number
  */
  let request = indexedDB.open("ExpenseDB", 1);

  /*
   onupgradeneeded runs only when:
   - Database is created first time
   - Version number changes
   Used to create objectStore (like table)
  */
  request.onupgradeneeded = function (event) {
    db = event.target.result;

    // Create objectStore if it doesn't exist
    if (!db.objectStoreNames.contains("expenses")) {
      db.createObjectStore("expenses", {
        keyPath: "id",          // Primary key
        autoIncrement: true    // Automatically increment ID
      });
    }
  };

  // Runs when database opens successfully

```

```

request.onsuccess = function (event) {
    db = event.target.result;
    viewExpenses(); // Automatically load stored data
};

// Runs if database fails to open
request.onerror = function (event) {
    alert("Database Error: " + event.target.errorCode);
};

/*
    INSERT Operation
    Adds new expense record
*/
function addExpense() {

    // Get input values
    let title = document.getElementById("title").value;
    let amount = document.getElementById("amount").value;
    let date = document.getElementById("date").value;

    // Basic validation
    if (title === "" || amount === "" || date === "") {
        alert("Please fill all fields");
        return;
    }

    // Create transaction (readwrite required for insert)
    let transaction = db.transaction(["expenses"], "readwrite");

    // Access objectStore
    let store = transaction.objectStore("expenses");

    // Create expense object
    let expense = {
        title: title,
        amount: amount,
        date: date
    };

    // Add record to store
    let addRequest = store.add(expense);

    // If insertion successful
    addRequest.onsuccess = function () {
        alert("Expense Added Successfully!");

        // Clear input fields
        document.getElementById("title").value = "";
        document.getElementById("amount").value = "";
        document.getElementById("date").value = "";

        viewExpenses(); // Refresh table
    };
}

```

```

// If insertion fails
addRequest.onerror = function (event) {
    alert("Insert Error: " + event.target.error);
};
}

/*
SELECT Operation
Fetch all records using getAll()
*/
function viewExpenses() {

    // readonly transaction for fetching data
    let transaction = db.transaction(["expenses"], "readonly");
    let store = transaction.objectStore("expenses");

    let getRequest = store.getAll();

    getRequest.onsuccess = function (event) {

        let expenses = event.target.result;
        let tableBody = document.querySelector("#expenseTable tbody");

        // Clear existing rows before reloading
        tableBody.innerHTML = "";

        // Loop through each expense
        expenses.forEach(function (expense) {

            // Create table row dynamically
            let row = `
<tr>
    <td>${expense.id}</td>
    <td>${expense.title}</td>
    <td>${expense.amount}</td>
    <td>${expense.date}</td>
    <td>
        <button onclick="deleteExpense(${expense.id})">
            Delete
        </button>
    </td>
</tr>
`;

            tableBody.innerHTML += row;
        });
    };

    getRequest.onerror = function (event) {
        alert("Select Error: " + event.target.error);
    };
}

/*
DELETE Operation

```

```

        Deletes record using ID
        */
        function deleteExpense(id) {

            let transaction = db.transaction(["expenses"], "readwrite");
            let store = transaction.objectStore("expenses");

            let deleteRequest = store.delete(id);

            deleteRequest.onsuccess = function () {
                viewExpenses(); // Refresh table after deletion
            };

            deleteRequest.onerror = function (event) {
                alert("Delete Error: " + event.target.error);
            };
        }

    </script>

</body>

</html>

```

OUTPUT:

The screenshot displays the 'Mini Expense Tracker (IndexedDB)' web application. On the left, the Chrome DevTools 'Application' and 'Storage' panels are visible. The 'Application' panel shows the 'ExpenseDB' database with a 'default' bucket, version 1, and one object store. The 'Storage' panel shows the 'IndexedDB' database with the 'Expenses' object store selected. The main browser window shows the application interface, which includes input fields for 'Expense Title', 'Amount', and 'Date' (formatted as 'dd-mm-yyyy'). Below these fields are 'Add Expense' and 'View Expenses' buttons. The 'Expense List' table displays a single entry with ID 2, Title 'Petrol', Amount '1000', Date '2026-02-23', and a 'Delete' button.